

Unidade 4 — Fundamentos de RAG

Aplicações de Modelos de Linguagem Natural

- 1 Introdução
- 2 Geração Aumentada por Recuperação (RAG)
 - O pipeline RAG

A necessidade de acessar informação

- A necessidade de acessar informação é um aspecto fundamental da atividade humana.
- As empresas são particularmente afetadas por isso: elas têm grandes volumes de dados disponíveis, e seus funcionários precisam acessá-los.
- Infelizmente, os sistemas de busca corporativos atuais não atendem bem a essa necessidade.

Agentes conversacionais baseados em LLMs

- Agentes conversacionais baseados em LLMs revolucionaram a forma de acessar informação, oferecendo duas vantagens em relação aos sistemas corporativos tradicionais:
 - ① Permitem que o usuário faça perguntas em **linguagem natural** e receba respostas igualmente conversacionais.
 - ② São cada vez mais capazes de lidar com **tarefas de busca complexas**, facilitando a resolução de problemas e a tomada de decisão.

Exemplos de uso

Pessoas já usam agentes conversacionais para obter receitas culinárias, resolver tarefas de química e problemas de geometria.

Dados internos das organizações e alucinação

- Nas organizações, a necessidade de informação costuma estar relacionada a **dados internos** que normalmente não estão disponíveis na internet — por exemplo, um resumo de uma análise de requisitos ou detalhes de um contrato.
- Como é pouco provável que um LLM “puro” (sem nenhum tipo de adaptação) tenha usado esses dados específicos durante seu treinamento, as respostas geradas para esse tipo de pergunta tendem a ser pouco confiáveis.
- De forma mais geral, os LLMs às vezes geram respostas sem nenhuma base factual, um fenômeno chamado de **alucinação**.

Definição

Alucinação: “conteúdo que é inconsistente com fatos do mundo real ou com as entradas fornecidas pelo usuário” (Ji et al., 2023).

RAG: uma resposta a essas limitações

- A **Geração Aumentada por Recuperação** (*Retrieval-Augmented Generation*, RAG) foi proposta como um novo framework de IA que busca integrar conhecimento adicional, como dados organizacionais.
- Com o RAG, o sistema gera respostas que podem ser vinculadas a esse conhecimento adicional.
- Isso permite que os usuários acessem informações internas da organização e reduz o risco de alucinações.
- Essa nova arquitetura representa um avanço importante em relação às arquiteturas anteriores.

1 Introdução

2 Geração Aumentada por Recuperação (RAG)

- O pipeline RAG

A ideia central do RAG

- A ideia central do RAG é combinar a **capacidade generativa dos LLMs** com **conhecimento externo** recuperado de uma base de dados separada (por exemplo, uma base de dados organizacional).
- Lewis et al. (2020) cunharam o termo “*Retrieval-Augmented Generation (RAG)*” e propuseram um framework geral que combina:
 - a força da **memória paramétrica** pré-treinada (o próprio LLM);
 - com uma **memória não paramétrica** (uma base de dados separada).
- Essa combinação foi proposta como uma nova forma de melhorar o desempenho em tarefas que exigem muito conhecimento.

Memória paramétrica

- A **memória paramétrica** é a informação armazenada nos parâmetros de um modelo.
- Em vez de armazenar diretamente dados prontos e significativos, a memória paramétrica guarda parâmetros do modelo que podem ser usados posteriormente para regenerar informação.
- Quanto mais parâmetros um modelo tem, mais informação ele consegue representar fielmente; por isso, os modelos atuais costumam incluir o número de parâmetros no próprio nome.
- É importante notar que a memória paramétrica **só contém informação que foi fornecida durante o treinamento**.

Exemplo

O **Mistral 7B** tem sete bilhões de parâmetros — o número está no próprio nome do modelo.

Memória não paramétrica (ou memória externa)

- A **memória não paramétrica** é informação que fica fora do modelo, por exemplo, informação vinda de uma base de dados.
- Por isso, essa informação é independente das limitações do modelo.
- Em outras palavras, a memória não paramétrica permite integrar conhecimento que não foi usado durante o processo de treinamento.

Exemplo

Usar documentos de política interna de uma empresa para melhorar o desempenho de um sistema de perguntas frequentes (FAQ).

Três formas de incorporar conhecimento a um LLM

Para entender melhor o RAG, é útil compará-lo com outras duas formas de adaptar um LLM a um contexto específico (Figura 1).

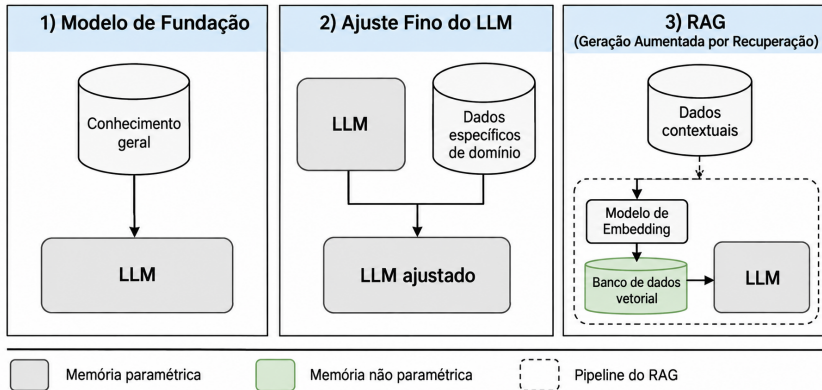


Figura 1: comparação entre as três abordagens para incorporar conhecimento a um LLM — modelo de fundação, ajuste fino (fine-tuning) e RAG.

Modelo de fundação e ajuste fino

- ❶ **Modelo de fundação (*foundation model*):** todos os dados de treinamento resultam em um LLM e passam a fazer parte do modelo paramétrico. Em geral, os modelos são treinados com enormes corpora de texto da internet.
 - A criação e o treinamento de um modelo de fundação exigem recursos e infraestrutura massivos, o que só é viável para organizações muito grandes; mas, uma vez criados, muitos desses modelos ficam disponíveis como serviço para qualquer pessoa usar.
 - Modelos de fundação são bem equipados para uma ampla variedade de aplicações que não dependem de dados organizacionais ou privados.
- ❷ **Ajuste fino do LLM (*fine-tuning*):** dados adicionais e específicos de um domínio (como documentos internos de uma empresa) são usados para atualizar os parâmetros do LLM, o que melhora seu desempenho para as tarefas e o domínio específicos em questão.
 - O ajuste fino é menos custoso e exige menos recursos do que treinar um modelo de fundação do zero, o que permite que organizações menores e até indivíduos ajustem modelos de fundação para um contexto específico.
 - *Exemplo:* treinar um modelo capaz de responder perguntas específicas do domínio da agricultura.

RAG como terceira abordagem

- 3 **RAG:** uma base de dados separada, com informação vetorial (*embeddings*), é criada a partir de um modelo de embeddings e de dados contextuais. Essa parte separada é a **memória não paramétrica**.
 - Esse banco de dados vetorial é usado para “aumentar” a entrada do LLM e assim melhorar seu resultado.
 - Criar um banco de dados vetorial é ainda menos custoso, em termos de recursos, do que fazer um ajuste fino; por isso, com capacidade tecnológica suficiente, essa abordagem é viável para muitas organizações.

Comparação entre as três abordagens

Modelo de Fundação → todos os dados de treinamento viram parte do LLM (memória paramétrica).

Ajuste Fino (*Fine-Tuning*) → dados adicionais do domínio atualizam os parâmetros do LLM já existente.

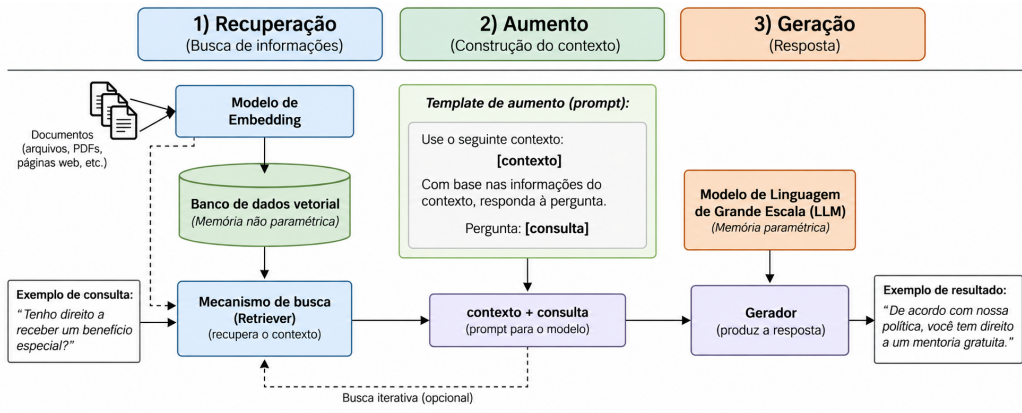
RAG → um banco de dados vetorial separado (memória não paramétrica) é criado e usado para complementar o LLM, sem alterar seus parâmetros.

Legenda: comparação entre as três formas de incorporar conhecimento a um LLM: modelo de fundação, ajuste fino (fine-tuning) e RAG.

O pipeline RAG: da consulta ao resultado

- O uso de uma arquitetura RAG resulta em um **pipeline** que começa com uma consulta (*query*) do usuário e termina com um resultado.
- Entre esses dois pontos, existem três partes fundamentais:
 - 1 **Recuperação** (*retrieval*);
 - 2 **Aumento** (*augmentation*);
 - 3 **Geração** (*generation*).

Um pipeline RAG básico



Modelo de embeddings e recuperador

- **Modelo de embeddings:** traduz dados de diferentes modalidades (texto, áudio, imagem ou vídeo) em um vetor numérico.
 - É essencial que o **mesmo modelo de embeddings** usado para criar o banco de dados vetorial seja também usado para transformar a consulta do usuário em vetor, já que a similaridade entre a consulta e os documentos (ou blocos de texto) da base é medida usando justamente esses vetores.
- **Recuperador (*retriever*):** busca a informação mais relevante no banco de dados vetorial, calculando um escore de similaridade entre o vetor da consulta de entrada e os vetores dos documentos armazenados.
 - Como resultado, o recuperador sugere um **contexto**, que normalmente é uma lista de blocos de texto (*chunks*) ou documentos recuperados.
 - Na prática mais comum, são selecionados os “**top k**” resultados (por exemplo, os 5 mais relevantes), ordenados pelo escore de similaridade — uma abordagem também chamada de “**princípio de ranqueamento por probabilidade**”.

Exemplo

Se a consulta menciona um cliente específico, os documentos mais relevantes relacionados a esse cliente são incluídos no contexto.

- **Aumento (*augmentation*)**: o contexto recuperado é usado para gerar uma consulta estendida. Para isso, os LLMs utilizam um tipo de modelo (*template*) de aumento que define como a consulta original do usuário deve ser combinada com o contexto.

Exemplo simples de template de aumento

“Use o seguinte contexto: [contexto]. Dado o contexto fornecido, responda à pergunta: [pergunta].”

- A parte [contexto] pode incluir ainda links para os documentos mais relevantes encontrados no banco de dados vetorial.
- **Gerador (*generator*)**: recebe a consulta já “aumentada” com a informação vinda do banco de dados e gera um novo resultado.
- Esse processo de geração também pode aproveitar a informação do contexto para fornecer, junto com a resposta, um link para o documento original (por exemplo, um hiperlink para o documento-fonte).

As três etapas do pipeline RAG

Fluxo do pipeline RAG

Consulta do usuário

- ↓ **Recuperação:** busca no banco de dados vetorial pelos blocos de texto mais relevantes.
- ↓ **Aumento:** a consulta original é combinada com o contexto recuperado, usando um modelo de aumento.
- ↓ **Geração:** o LLM produz a resposta final com base na consulta aumentada.
- ↓ **Resultado**

Legenda: as três etapas fundamentais do pipeline RAG: recuperação, aumento e geração.